The Frame Is the Token: Generative Videogames on a Native Inference Stack

Hanzo AI — ML Systems

Games crossover in the native-inference series — an inference-systems reading of the playable generative game

Abstract

A playable generative game is not a rendering problem wearing a neural coat; it is an *inference-systems* artifact. Strip away the framing and what remains is an action-conditioned world model run in a *fixed-shape frame loop* — generate frame $t{+}1$ from a window of past frames and the player's action, exactly as an autoregressive decoder generates token $t{+}1$ from its context — wrapped in a set of asset-generation pipelines (3D, music, voice, video, texture) and an authoring surface. Our thesis is that this structure is best served by a *single native engine*, not a zoo of Python microservices: on the frame critical path, a 33–66 ms budget cannot absorb an HTTP hop, a second CUDA context, or a cold model load, and the three costs a service mesh imposes — latency, resident memory, and operational surface — are precisely the three a real-time neural game cannot pay. We develop the thesis in four measured moves. First, the **latency budget**: interactive play at 15 fps–30 fps allots 33–66 ms per frame, and the frame pipeline splits cleanly into the same two regimes our inference work already characterizes — the per-frame denoise loop over a compact latent is a fixed-shape, launch-overhead-bound computation, the CUDA-graph-favorable regime that won ≈1.47× on decode, while the VAE encode/decode at frame resolution is a compute-bound convolution wall (measured 96.4% GPU utilization on dense prefill, NVIDIA GB10, where a graph recovers nothing) whose proven cure is a tiny distilled encoder, the same TAESD move that carried our native lip-sync dub to 11.6 fps combined (f16, GB10). *Which regime a stage is in dictates which lever moves it* — and that distinction is the section's intellectual content. Second, the **pipeline matrix**: modality by modality, honestly labelled measured-live (LLM game logic, TTS/voice, and a native localization dub measured at LSE-C 7.05, speaker cosine 1.0), in-port (3D/TRELLIS-class, video understanding), or planned (the world model itself; music; video generation). Third, the **distributed angle**: a world model too large for one consumer node splits pipeline-parallel across a heterogeneous home cluster over the series' canonical-`f32` ring (measured cross-vendor 50.4 tok/s decode, AMD gfx1151 + NVIDIA GB10), the neural-game analog of a render farm — with single-player as the latency-bound stream and spectate/multiplayer as batched rollouts in the throughput regime. Fourth, **authoring**: node-graph editing against a *persistent* world-model session is prefix-cache reuse — editing the game while it runs swaps the conditioning suffix against a cached world-state prefix. We close with game-specific **open problems** that we do not duplicate from the series' open-problems companion but cross-reference and extend — centrally, that client-side prediction and speculative decoding are *the same algorithm*: a cheap local draft rolled forward optimistically and reconciled against an authoritative target, with rollback on disagreement. Because the frame is the token, that isomorphism is exact, and the open question it raises — the acceptance criterion for a *perceptual* rather than a distributional match — is, to our knowledge, new. Frame numbers for a world model on our stack are labelled projection; the primitives they rest on are labelled measured.

1 Thesis: a playable generative game is an inference-systems artifact

The recent generation of neural game engines — GameNGen simulating DOOM from a diffusion model [8], Oasis generating Minecraft frame-by-frame from keyboard input [9], Microsoft's Muse/WHAM [10] and MineWorld [12], Skywork's Matrix-Game [11], and DIAMOND's diffusion world model [13] — is usually read as a graphics result: the network has learned to *render*. That reading buries the systems structure. Look at the loop these models actually run and it is an inference loop, not a render loop:

$$\text{frame}_{t+1} \;=\; f_\theta(\text{frame}_{t-w+1:t},\ \text{action}_t;\ \text{state}_t)\,, \tag{1}$$

a function of a bounded window of past frames and the current action, advanced one step at a time, forever. This is *autoregressive generation with a fixed context window*, and it is the same shape as language decode: $\text{token}_{t+1} = g_\theta(\text{token}_{t-w+1:t})$. The frame is the token. The action is the extra conditioning. The window of past frames is the KV cache. Once this identification is made, the entire apparatus a native inference engine was built for — fixed-shape kernel loops, CUDA-graph capture, KV-cache reuse, speculative decoding, pipeline parallelism, prefix caching — transfers to the game loop not by analogy but by *identity of the computation*. This paper is the systematic working-out of that transfer.

The whole artifact, not just the world model. A shippable game is more than Eq. (1). It needs 3D assets (meshes, materials), music and ambient audio, character voices, in-engine video, textures, an LLM driving NPC dialogue and game logic, and an authoring tool in which a designer builds and edits all of it. Every one of these is, today, a generative-model inference: TRELLIS-class image-to-3D [14], ACE-Step-class music [15], neural TTS and a lip-sync localization dub, Wan-class video [16], an image model for textures, and a language model for logic. The naive architecture stands each behind its own Python service. We argue that architecture is wrong for a game, and the argument is a systems argument with three axes.

Latency. The world-model loop has a hard real-time deadline (§2): at 30 fps, 33.3 ms per frame, end to end. A microservice boundary on that path — serialize the latent, HTTP to the VAE service, deserialize, copy across a second process's CUDA context — spends milliseconds the budget does not have, per frame, forever. The asset pipelines are not on the per-frame path, but they are on the *authoring* path (§5), where a designer's edit-to-preview loop wants to feel immediate. A single engine keeps the world-model session, the VAE, the samplers, and the asset models in one address space behind one CUDA context, so nothing on the frame critical path crosses a process boundary. The cross-process copy is not slow because the network is slow; it is slow because it exists at all, once per frame.

Memory. A per-modality service mesh loads redundant state. Each Python process carries its own framework runtime (a bare PyTorch process is gigabytes of RSS before a single weight loads), its own CUDA context, and its own copy of components that are in fact *shared* — the SD-family VAE that the world model, the 3D pipeline, the video pipeline, and the texture model all decode through is one autoencoder, not four. On a consumer accelerator — a 24 GB discrete card or a 128 GB unified-memory APU, the exact targets our engine is built for [1] — loading it once versus four times is the difference between the game fitting and not. One engine deduplicates shared weights by construction.

Operational surface. N services are N deployment units, N crash domains, N version skews, N ports to secure. A generative game cannot tolerate partial availability: if the VAE decoder is the service that OOM-restarted, there is no frame to show — the game is down, not degraded. The engine's answer is the series' answer to every other deployment: one binary, built once from one source, lowering to CUDA, ROCm, Metal, Vulkan, and CPU behind a single loader [1, 2]. The game inherits that operational surface — one process, one artifact, one thing to watch — rather than paying a mesh's.

We are precise about what is and is not measured. The three-axis argument above is an *architectural* claim; we have not run a head-to-head of a full game against a Python service mesh, and we do not report one. What we report measured are the *primitives* the claim rests on — the two graph regimes (§2), the dub pipeline (§3), the cross-vendor ring (§4) — each on labelled hardware. Where we carry a primitive forward to a world model we have not yet run, we label it projection. The discipline of the series [5, 7] — measured, designed, projected, never blurred — is the discipline here.

2 The latency budget

This is the core systems section, and its thesis is a partition: the frame pipeline is not one workload with one bottleneck but *two workloads in two different performance regimes*, and the regimes are exactly the two the inference series already named — the launch-overhead-bound decode regime, where CUDA-graph capture pays, and the compute-bound prefill regime, where it cannot. Getting a generative game to frame rate is the problem of putting each stage in the right regime and pulling the lever that regime responds to.

2.1 The arithmetic

Interactive play needs 15 fps–30 fps to feel like control rather than a slideshow, 60 fps to feel native. The per-frame wall-clock budget is the reciprocal:

$$T_{\text{frame}} \;=\; T_{\text{ingest}} + T_{\text{enc}} + K \cdot T_{\text{step}} + T_{\text{dec}} \;\le\; \frac{1}{\text{fps}} \quad\Longrightarrow\quad \begin{cases} 66.7\,\text{ms} & \text{at } 15\,\text{fps} \\ 33.3\,\text{ms} & \text{at } 30\,\text{fps} \\ 16.7\,\text{ms} & \text{at } 60\,\text{fps}. \end{cases} \tag{2}$$

The terms: T_{ingest} embeds the action (a keypress, a mouse delta, a controller state) into the conditioning — negligible, a small matmul. T_{enc} encodes the previous frame to the latent the model works in, if the model is pixel-conditioned rather than latent-recurrent. $K \cdot T_{\text{step}}$ is the generative core: K denoise steps of a diffusion world model, or one autoregressive pass over the frame's tokens, at T_{step} each. T_{dec} decodes the produced latent back to pixels. The budget is brutal and it drives every design choice in the field: it is why GameNGen distills to 4 sampling steps [8], why Oasis and MineWorld push few-step and diagonal decoding [9, 12], and why real-time world models live or die on step-count. K is the term you fight.

2.2 Two regimes, one pipeline

The intellectual content is not the arithmetic; it is recognizing that the two dominant terms sit in *opposite* regimes, so they respond to opposite levers.

The denoise loop $K \cdot T_{\text{step}}$ is decode: launch-overhead-bound, and graph-favorable. A single denoise step over a compact latent (e.g. a 32×32×4 or 16×16 token grid) is a long chain of *small* kernels — projections, attention over a short frame-token sequence, group-norms, elementwise activations — each finishing in microseconds and gated by launch and scheduling overhead, not arithmetic. Crucially, *the shape is fixed*: same resolution, same latent dimensions, same K, same kernel sequence, every frame. This is precisely the property that makes language *decode* capturable — one token, one static set of kernels — and the series measured a ≈1.47× decode speedup from capturing that static submission into a CUDA graph and replaying it [1, 6]. The frame loop is *more* graph-favorable than decode, not less: it has no data-dependent control flow at all, whereas even decode must occasionally handle a variable-length context. The contrast that proves the point is the mixture-of-experts *prefill* wall the series dissects [7]: there, graph capture fails because MoE routing is data-dependent — which experts fire changes per step, so the shape changes per step, and a static graph cache thrashes. A dense, fixed-topology world-model step has no such pathology. It is the clean case for capture, and the game loop lives in it by construction.

The VAE encode/decode $T_{\text{enc}}, T_{\text{dec}}$ is prefill: compute-bound, and graph-immune. The autoencoder runs at *frame* resolution, not latent resolution: its convolutions are large GEMMs over a 256×256 or 512×512 image, and they saturate the device. This is the compute-bound regime the series characterizes on dense prefill, where the engine measured ≈96.4% GPU utilization on an 8B-parameter dense model (bf16, flash attention, NVIDIA GB10, node *spark*), computed as the union of CUDA-kernel intervals over the wall span so it is not a stream-overlap artifact [7]. When a stage is 96% compute-bound there is at most ≈3.6% of idle interstice for a graph to remove — graph capture is the wrong lever because there is nothing for it to recover. We verified the general form of this directly in the series: capturing a whole dense prefill as one fixed-shape submission reproduces the eager logits bit-for-bit and moves throughput by less than run-to-run noise [7]. The VAE wall is the same shape of problem.

The measured cure for the VAE wall: a tiny distilled encoder. Because the VAE is compute-bound, its lever is *less computation*, not less overhead: a smaller autoencoder. We have measured exactly this lever on the adjacent workload. In the native lip-sync localization dub (§3), profiling identified the VAE *encoder*'s high-resolution convolution GEMMs as the realtime wall — not batching, not GroupNorm, not host-device transfer, all measured and ruled out — and the proven fix was a TAESD-style tiny distilled encoder mirroring the tiny decoder that Stable-Diffusion pipelines already ship [17, 18], which moved the combined pipeline to 11.6 fps (f16, node *spark*/GB10). The same move applies to a pixel-conditioned world model: replace the full VAE on the frame path with a distilled encoder/decoder pair, trading a small, bounded reconstruction loss for the order-of-magnitude compute reduction the budget demands. A world model that keeps its state in latent space and only *decodes* to pixels pays T_{dec} but not T_{enc}, which is why latent-recurrent designs have a structural budget advantage and why the pixel-conditioned designs need the tiny encoder most.

The partition is the design rule. Table 1 states the rule the section exists to establish. Do not reach for one lever and apply it everywhere; identify each stage's regime and pull that regime's lever. Capture the denoise loop into a graph; shrink the VAE with distillation and fuse its kernels. The two are composable — graph the launch-bound half, compute-optimize the compute-bound half — precisely *because* they are in different regimes and do not contend for the same lever.

Frame-pipeline stage	Regime	Bottleneck	Correct lever
Action ingest	trivial	—	(matmul; ignore)
VAE encode T_{enc}	prefill-like	compute-bound	tiny distilled encoder (TAESD); kernel fusion
Denoise loop $K \cdot T_{\text{step}}$	decode-like	launch-overhead	CUDA-graph capture (fixed shape); step distillation to cut K
VAE decode T_{dec}	prefill-like	compute-bound	tiny distilled decoder; fusion

Table 1: The frame pipeline is two regimes, not one. The denoise loop is fixed-shape and launch-overhead-bound — the same regime where CUDA-graph capture won ≈1.47× on decode [1] and where the data-dependent MoE-prefill wall does *not* apply, because a dense world-model step has no data-dependent routing [7]. The VAE runs at frame resolution and is compute-bound — the regime where dense prefill measured 96.4% utilization and a graph recovers nothing [7], cured instead by a smaller (distilled) autoencoder, the measured TAESD move from the native dub. Choosing the lever by regime is the whole discipline.

2.3 The frame-latent cache is the KV cache

Eq. (1) conditions frame $t{+}1$ on a window of the last w frames. A correct implementation does not recompute the representation of those w frames every step; it computes each frame's contribution once, caches it, and reuses it for every subsequent frame until it slides out of the window — which is exactly the token KV cache, byte for byte the same mechanism. The consequences transfer wholesale. The *context horizon* w is the KV window: what the model can condition on, and therefore what it can keep consistent, is bounded by it, and everything past w is forgotten (§6, the persistence problem). *Sliding-window eviction*, *KV compression* (an FP8 frame-latent cache halves the resident state, with the cross-hop error-accumulation caveat the series raises for migration [7]), and *prefix reuse* (§5) all apply to the frame cache with no reinterpretation. A native engine that already implements KV-cache management for tokens implements frame-latent caching for free; a per-service architecture, where each request is stateless, cannot — it has nowhere to keep the cache between frames. This is the sharpest single instance of the thesis: the persistent, in-process session is not a convenience, it is the only place the frame cache can live.

3 The pipeline matrix

A game is a portfolio of generative modalities, and honesty about the portfolio is a systems virtue: the value of one engine is that a modality's status is a *loader* question, not an *architecture* question — adding music is loading a music model, not standing up a service. Table 2 states, per modality, what is measured-live on the native stack today, what is in-port, and what is planned, with no rounding up.

Measured-live. *LLM game logic* is the engine's core competence: language decode at `llama.cpp` parity on every backend [1], driving NPC dialogue, quest state, and rule adjudication in-process. *Voice* — ASR for player speech and neural TTS for characters — runs native. The strongest measured cell is the *localization dub*: an ASR → translate → TTS → lip-sync pipeline that runs 100% native (no Python on the inference path), measured at lip-sync confidence LSE-C 7.05 — beating the PyTorch reference implementation it was ported from — with speaker-identity cosine

Modality	Role in the game	Status	Native basis / port target
LLM game logic	NPC dialogue, quests, rule adjudication	measured-live	decode parity, every backend [1]
Voice (ASR + TTS)	player speech in, character speech out	measured-live	native speech path
Localization dub	lip-synced multilingual voice	measured-live	LSE-C 7.05, cosine 1.0, 11.6 fps (GB10)
Image / texture gen	materials, 2D assets, textures	measured-live	native image diffusion
3D assets	meshes, props, environments	in-port	TRELLIS-class latents [14]
Video understanding	playtest analytics, QA	in-port	native vision; temporal head stub
World model	**the playable frame loop**	**planned**	**graph loop + KV + tiny VAE (§2)**
Music / ambient	score, adaptive audio	planned	ACE-Step-class [15]
Video generation	cutscenes, in-engine video	planned	Wan-class MoE [16]

Table 2: The modality portfolio on the native stack, labelled without rounding up. Measured-live cells carry first-party numbers on labelled hardware; in-port cells have native primitives running but the modality's specific head incomplete; planned cells share diffusion machinery with live modalities but are not yet running. The world model — the central artifact — is honestly planned: §2 projects its budget from measured primitives, not from a measured world model.

1.0 and the 11.6 fps combined throughput (f16, node *spark*/GB10) that the tiny-encoder work of §2.2 produced. A dub is a game-shaped workload in miniature: a diffusion UNet, a VAE on the critical path, an audio model, and a hard latency target, all native — it is the existence proof for the frame pipeline's components.

In-port. *3D assets* (TRELLIS-class structured-latent image-to-3D [14]) reuse the diffusion-UNet-plus-VAE machinery the dub already exercises natively; the port target is the structured-latent representation and its meshing, not new infrastructure. *Image generation* for textures and 2D assets is live. *Video understanding* for playtest analytics — summarizing sessions, detecting stuck states, scoring difficulty from recorded play — rests on the native vision path, which runs; the video-specific temporal head is a stub, so we mark the modality in-port rather than live.

Planned. The *world model* itself — the artifact of §2 — is not yet running on the stack; what exists are the primitives (fixed-shape graph loops, KV reuse, native VAE) it will compose. We say plainly: no world-model frame rate on our engine is measured, and §2's budget for it is projection built on measured primitives. *Music* (ACE-Step-class diffusion with a deep compression autoencoder [15]) and *video generation* (Wan-class MoE video diffusion [16]) are planned ports; the diffusion core is shared with modalities already native, which is the whole point of the portfolio-on-one-engine argument — but shared machinery is not a running model, and we do not claim it as one.

4 The distributed angle: a render farm for neural game engines

The frontier world models are large — Matrix-Game is a 17B-parameter model [11] — and the largest will not fit on one consumer node. The series already built the substrate for splitting a model across a heterogeneous fleet, and it transfers directly: a home cluster of mixed accelerators (an AMD APU, an NVIDIA dev kit, a Mac) is a game studio's render farm re-imagined for a neural engine, and the same *canonical-`f32` ring* that carries cross-vendor LLM inference carries a cross-vendor world model.

The measured substrate. The series demonstrated one engine, built from one source for three vendors' kernels, sharding a model *tensor-parallel* across an AMD/NVIDIA boundary over a commodity 2.5 GbE link and decoding coherently at 50.4 tok/s (TTFT 0.09 s, 213 tok/s prefill; AMD gfx1151 node *evo* + NVIDIA GB10 node *spark*) [5]. The load-bearing fix was a canonical-`f32` wire: the two ranks legitimately run different 16-bit compute dtypes (f16 on the ROCm rank, bf16 on the CUDA rank), and only a lossless `f32` payload on the wire lets them agree, reduced with a canonical tie-break so a rounding difference cannot fork the streams. The same work makes the case, on a measured $\approx 3.3\times$ synchronization tax, that *pipeline parallelism* — contiguous layer ranges, activations-only across each node boundary, one generation loop — is the structurally right primitive for a heterogeneous cluster [5, 6], and the pipeline-parallel keystone is measured on the same *evo*+*spark* pair at 145.8 tok/s decode on a 0.6B model and 45.2 tok/s on a 30B-A3B mixture-of-experts model.

Why it fits the game. These numbers are LLM decode, not world-model frames; carrying them to a world model is projection, and we label it so. But the *structure* carries exactly, because the frame is the token. Pipeline parallelism ships an activation tensor across each seam, not a weight slice, so it is format-agnostic and sidesteps quant-aware sharding [7]; a world model's frame latent is just such an activation. And the two play modes map onto the two regimes the series separates:

- **Single-player is the latency-bound stream.** One player, one autoregressive frame sequence, each frame on the critical path of the next — this is the single-stream decode regime, governed by the geo-distributed latency floor $1/(C + T_{\text{ring}})$ the series derives [6]: minimize per-frame latency, keep the ring turn co-located, keep the frame cache node-local. A cross-continent ring floors a single stream at a few frames per second regardless of accelerator speed, which is the physics case for keeping a single-player world model regional, and the case for the speculative rollout of §6.
- **Spectate and multiplayer are the throughput regime.** Many concurrent players, or many spectators watching one world, are many rollouts — and a spectator, not in control, does not need low action-latency, so their rollouts can be *batched* and run at higher per-frame latency for throughput. This is the micro-batched pipeline-parallel regime: fill the pipe, amortize the weight load across the batch, run the throughput dual of the layer-assignment optimization [7]. The latency-bound single stream and the throughput-bound batch are the same two regimes, one primitive, that organize the whole series.

5 Editing while it runs: authoring as prefix reuse

A generative game needs an authoring surface — a node-graph editor (Hanzo Studio) in which a designer wires prompts, conditioning, LoRA adapters, reference assets, and rules, and drives the

native endpoints behind each node. The systems content of authoring is the *edit-while-running* loop, and it is another exact instance of an inference primitive: prefix caching.

The edit loop is a prefix swap. A persistent world-model session holds accumulated state — the frame-latent KV cache of §2.3, the current world configuration — as a *prefix*. An edit (swap a LoRA, change a conditioning prompt, drop in a new asset, adjust a rule) changes the *suffix* of the conditioning, not the accumulated world state. The naive implementation tears down the session and re-simulates from scratch on every edit; the correct one keeps the world-state prefix resident and recomputes only the changed conditioning — exactly prefix caching in LLM serving, where the KV for a shared prompt prefix is retained across requests and only the divergent suffix is recomputed [6]. Editing the game while it runs *is* hot-swapping the conditioning suffix against a cached world-state prefix, and the designer sees the change take effect on the live world in the next frame rather than after a reload. This is only possible with a persistent in-process session: a stateless per-request service has no prefix to keep, so every edit is a cold start. The authoring loop is thus the second place (after the frame cache, §2.3) where the persistent native session is not an optimization but a precondition, and it connects authoring latency to the same session/prefix-reuse machinery the engine already runs for chat.

6 Open problems specific to generative games

The series' open-problems companion [7] enumerates eight problems of *heterogeneous distributed inference*; we do not restate them. The problems here are the ones the *game* adds — they sit on top of that agenda, cross-reference it where they touch, and are stated to make the open question precise rather than to claim a solution. The centerpiece, developed first and at length, is an isomorphism we believe is new and exactly true.

6.1 Action-latency compensation: speculative decoding *is* client-side prediction

The two mechanisms, and the claim. *Speculative decoding* [19, 20] accelerates autoregressive generation: a cheap *draft* model proposes K tokens; the expensive *target* model verifies all K in one batched forward pass; the longest agreeing prefix is accepted and, on the first disagreement, the target's own distribution is sampled and the rest discarded — yielding output distributed *exactly* as the target would produce, at fewer target invocations, paying the target's latency once per accepted *block* instead of once per token. *Client-side prediction* [21], the netcode technique every real-time multiplayer game ships, hides server round-trip latency: the client runs a cheap local simulation to predict the next frames and shows them immediately; when the authoritative server state arrives, agreement confirms the prediction and disagreement triggers a rollback-and-replay from the corrected state (the visible "rubber-band"). Our claim is that these are not analogous — they are the *same algorithm*, optimistic execution of a cheap predictor reconciled against an authoritative source with rollback on disagreement, and that because the frame is the token (§1) the identification is exact, term for term (Table 3).

Why the game instance is the interesting one. In a distributed world model (§4), the authoritative target may sit across a slow ring whose single-stream floor is a few frames per second [6] — unplayable. Speculative frame rollout is the cure the series already prescribes for tokens, read for frames: run a small, fast world model *locally* (on the player's own device) to roll frames forward under the player's actions and display them at once, hiding the ring latency; the large authoritative

Speculative decoding [19]	**Client-side prediction** [21]	**Speculative frame rollout (this paper)**
draft model (cheap, local)	client predictor (cheap, local)	small local world model (cheap, on-device)
target model (expensive, authoritative)	server (authoritative, remote)	large world model (authoritative, on the ring §4)
propose K tokens	predict K frames ahead	roll K frames forward under player actions
verify K in one batched pass	server reconciliation	authoritative model verifies K frames in one batched forward
accept prefix / resample on reject	confirm / rollback + replay	confirm frames / rubber-band correct
acceptance rate α	prediction hit rate	frame-agreement rate
distribution-exact	server-authoritative-exact	*perceptually* acceptable (the open relaxation)

Table 3: The isomorphism, term by term. Speculative decoding and client-side prediction are one algorithm — optimistic execution of a cheap local draft against a slow authoritative target, reconciled with rollback — and because the frame is the token the mapping is exact. The one row that is not a clean identity is the last, and it is exactly where the open problem lives (§6.1, "what is open").

model verifies the rolled-forward frames a block at a time; when they agree, the player saw the correct future with zero perceived latency, and when they disagree — a genuine surprise the small model could not predict — the world rubber-bands to the authoritative frame. This is speculative decoding's $r_{\text{spec}}(\alpha, K) = \frac{1-\alpha^{K+1}}{1-\alpha}/(Kc_d + C + T_{\text{ring}})$ speedup [6, 19], applied to frames, and it is why a distributed neural game can be playable at all across a slow link.

What is open: the acceptance criterion for a perceptual match. The isomorphism is exact except in the row that decides everything. Speculative decoding's accept test is a *distributional* identity — the drafted token is accepted with a probability that makes the output law identical to the target's, and a recomputed logit is comparable bit-for-bit only because the series' canonical-`f32` substrate makes it so (the determinism-boundary problem, §1 of [7]). A *frame* is a point in a continuous latent space, and two world models will essentially never produce the bit-identical frame; requiring a bit-exact match would drive the acceptance rate to zero and destroy the method. The frame accept test must therefore be *perceptual*: accept the drafted frame if it is perceptually close enough to the authoritative one that the correction would not be seen. This both *loosens* the test — a perceptual tolerance admits far more than a bit-exact one, so achievable α can be high — and makes soundness *ill-defined*: what metric (Δ in latent space, LPIPS, a learned perceptual head), at what threshold, bounds the *perceptible* rubber-band without either rejecting acceptable frames (stutter) or accepting drift (the world silently diverging from authority)? This is the open problem, and it is distinct from the companion's determinism boundary: there, agreement must be *bit-exact* for KV coherence; here, agreement must be *perceptual* for playability, and the relaxation from one to the other is the whole research question. Formally: define the frame acceptance predicate $\mathcal{P}(\hat{x}, x^\star)$ and bound, as a function of its threshold, both the expected accepted block length (hence the speedup) and the worst-case perceptible correction (hence the visual quality). Neither the token literature (which has no perceptual slack) nor the netcode literature (which reconciles *known* authoritative state, not a generative model's sample) answers it.

6.2 Consistency and persistence beyond the context horizon

The frame cache is the KV cache (§2.3), and its window w is a hard horizon: a world model conditions only on the last w frames, so a room you looked away from for longer than w is, to the model, unvisited — it will regenerate it inconsistently, the well-known failure of long-horizon world models. This is *not* the companion's KV-compression-for-migration problem [7], which is about moving an existing cache across nodes without error accumulation; it is about representing world state that must persist *beyond any KV window at all*. The open problem is an external, addressable *world-state memory* that the frame loop reads from and writes to — a persistent store of "what is where" (geometry, object state, what the player did) that re-conditions the model when a region re-enters the window, so consistency is bounded by the store's fidelity rather than by w. Its sub-questions: what is the right representation (an explicit 3D scene graph, a set of retrievable latents, a learned memory)? what is the read/write protocol on the frame critical path, and does it fit the budget of §2.1? and how does it compose with the speculative rollout of §6.1, whose local draft must consult the *same* memory to predict a re-entered region correctly? This is the game's version of long-context, and the horizon it must beat is spatial and temporal, not just sequence length.

6.3 Determinism for shipped games

A shipped game must be reproducible: the same seed and the same inputs must yield the same asset and the same playthrough, across driver versions, across the vendor backends the engine targets, and across the years a game is supported — both for the player's expectation that a world is stable and for QA to reproduce a bug. But GPU reductions are not associative across hardware, and the series' own work shows two ranks on two vendors diverge without a canonical-`f32` discipline [5, 4]. The open problem is a determinism contract for a shipped generative game: which computations must be pinned to the canonical-`f32` bit-exact path [4] (asset generation at author time, where reproducibility is worth the speed) and which may run fast and non-deterministic (the real-time frame loop, where perceptual tolerance §6.1 already admits variation)? This is the shipping dual of the companion's replicated-state determinism boundary: there, determinism is needed for two live ranks to *agree*; here, for one machine to *reproduce* another, possibly a driver generation later. The bit-exactness oracle [4] is the mechanism; the game-specific question is where to spend it, since pinning the whole frame loop to bit-exact would forfeit the graph and fusion wins of §2.

6.4 Provenance and license of generated assets

Every shipped asset in a generative game is a model output, which makes shipping one a provenance and licensing problem the series' correctness work does not touch. Each asset must carry *which* model produced it, under *which* weights and license, from *which* prompt — because the models in the portfolio (§3) do not share license terms: TRELLIS, MineWorld, Matrix-Game, and Oasis are MIT [14, 12, 11, 9], ACE-Step and Wan are Apache-2.0 [15, 16], but WHAM is released under a research-only Microsoft Research License [10], and a world model trained on a proprietary game's footage inherits that game's rights. A shipped title mixing assets from several is a derivative-works graph that must be tracked, not asserted. The open problem is a provenance-carrying asset pipeline: bind a signed manifest (C2PA-style [22], as our media work already does for dubbed audio) to every generated asset at generation time, propagate it through the authoring graph (§5), and gate the ship on a license check — so that "can we ship this asset commercially" is a query against recorded provenance, not a lawyer's reconstruction. This is an infrastructure problem the

single-engine architecture is well-placed to solve, because provenance is captured at the one point every asset passes through.

6.5 Evaluation metrics for playability

The field measures frame fidelity — FID, PSNR, LPIPS — but a game is not judged on how real a frame looks; it is judged on whether it is *playable*. A world model can score well on per-frame image metrics and be unplayable: unresponsive to actions, temporally incoherent across the horizon (§6.2), or simply not fun. The open problem is a playability metric — *action-consistency* (does the world respond correctly and promptly to the control?), *temporal coherence* (does state persist and evolve plausibly?), *controllability* (can a policy achieve intended goals in the generated world?), and the harder qualitative axis. This matters for the systems work because it is the objective the levers of §2 optimize *against*: step-distillation to cut K, tiny encoders, and speculative rollout all trade fidelity for latency, and without a playability metric there is no principled way to set that trade — how much frame quality may be spent to hit 30 fps before the game stops being fun is unanswerable until playability is measurable. It is the open problem that the rest depend on to know when they are done.

7 Landscape: open world models

Table 4 places the paper against the open world-model landscape, with every license verified from the project's Hugging Face model card or GitHub `LICENSE` rather than asserted from memory; where a project does not disclose a figure we mark it n/d rather than guess. The reference point is closed: GameNGen, the DOOM-simulating diffusion engine that opened the current wave [8], released neither weights nor code — as does the broader closed frontier (the DeepMind Genie line of foundation world models) — so it anchors the table as the thing the open models are measured against, not a thing one can build on. Among the open releases, the permissive-license cluster is real and usable: Oasis (MIT) [9], MineWorld (MIT) [12], Matrix-Game (MIT) [11], and DIAMOND (MIT) [13] ship weights and code under terms a commercial game can build on, while Microsoft's Muse/WHAM ships weights under a research-only license [10] that a shipped product cannot — the exact provenance distinction §6.4 argues must be tracked per asset. The systems reading of the table: these are all the same computation (Eq. (1)), differing in scale, tokenizer, and sampler, and every one of them is a fixed-shape frame loop that the two-regime discipline of §2 and the distributed substrate of §4 apply to unchanged. The engine does not care which of them it loads; that is the point.

8 Conclusion

A generative game reads as a graphics achievement and *is* an inference-systems artifact: an action-conditioned world model run in a fixed-shape frame loop — the frame is the token — surrounded by a portfolio of generative-asset pipelines and an authoring surface, all of which are model inferences. Reading it that way is not a rhetorical move; it is what lets the machinery a native inference engine already has — CUDA-graph capture, KV-cache reuse, pipeline parallelism over the canonical-`f32` ring, prefix caching, speculative decoding — transfer to the game by identity of computation rather than by analogy. The systems content is the two-regime partition of the frame budget: the denoise loop is fixed-shape and launch-overhead-bound, so it is graph-favorable exactly as decode is, while the VAE is compute-bound at frame resolution, so it wants a smaller network exactly as

Model	Origin	Params	License	Domain / notes
GameNGen	Google Research, DeepMind, Tel Aviv U.	SD-1.4 core (n/d)	closed (no release)	DOOM; 20 fps on TPU; *reference point* [8]
Oasis (open-oasis)	Decart + Etched	500M (open ckpt)	MIT	Minecraft; diffusion-transformer, action-conditioned [9]
Muse / WHAM	Microsoft Research	200M, 1.6B	MS Research (non-commercial)	Bleeding Edge; AR token model [10]
Matrix-Game	Skywork AI	17B (v1)	MIT	Minecraft/UE; keyboard+mouse diffusion I2W [11]
MineWorld	Microsoft Research	300M / 700M / 1.2B	MIT	Minecraft; visual-action AR transformer [12]
DIAMOND	Alonso et al. (NeurIPS'24)	RL-agent scale	MIT	Atari + CS:GO; diffusion world model [13]

Table 4: Open world models, licenses verified from each project's Hugging Face model card or GitHub `LICENSE` (not asserted from memory); undisclosed figures marked n/d. GameNGen is the closed reference the open field is measured against. The permissive (MIT/Apache) releases are buildable-on; Muse/WHAM's research-only license is not — the provenance distinction §6.4 requires be tracked per shipped asset. Every row is the same fixed-shape frame loop (Eq. (1)), which is why one engine serves all of them.

the dense-prefill wall did — and choosing the lever by regime is the discipline that gets a game to frame rate. The single-engine architecture is not an aesthetic preference; on the 33–66 ms frame path, a persistent in-process session is the only place the frame cache and the edit-loop prefix can live, and the memory and operational-surface costs of a service mesh are ones a real-time neural game cannot pay. We reported the primitives measured — the graph regimes, the native dub, the cross-vendor ring — and labelled the world model itself honestly as planned, its budget a projection over those primitives. And we set the game-specific open problems on top of, not duplicating, the series' agenda [7]: centrally, that client-side prediction and speculative decoding are one algorithm, so a distributed neural game is playable across a slow ring by speculating frames locally and reconciling them against the authority — with the acceptance criterion for a *perceptual* rather than a distributional match the open question the isomorphism newly poses. When the frame is the token, a game is a decode loop with a controller on it, and the whole native stack is already built for decode.

References

[1] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators: Measured AMD, NVIDIA, and Apple Performance, and hanzo-engine at llama.cpp Decode Parity on Every Backend.* Paper 05 (measured).

[2] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL — Thesis and Guided Tour.* Paper 07 (umbrella).

[3] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law.* Paper 07.1.

[4] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law.* Paper 07.4.

[5] Hanzo AI — ML Systems. *One Cluster, Every Vendor: Cross-Vendor Tensor-Parallel LLM Inference over Commodity Ethernet.* Distributed-inference companion (measured).

[6] Hanzo AI — ML Systems. *The Latency Physics of Geo-Distributed Autoregressive Decode.* Distributed-inference companion (analytical).

[7] Hanzo AI — ML Systems. *Open Problems in Heterogeneous Distributed Inference.* Distributed-inference companion (research agenda).

[8] D. Valevski, Y. Leviathan, M. Arar, S. Fruchter. *Diffusion Models Are Real-Time Game Engines (GameNGen).* arXiv:2408.14837, 2024.

[9] Decart and Etched. *Oasis: A Universe in a Transformer* (open-oasis; `Etched/oasis-500m`, MIT). 2024.

[10] A. Kanervisto et al. *World and Human Action Models towards gameplay ideation (Muse/WHAM).* Nature 638, 2025. Weights: `microsoft/wham`, Microsoft Research License.

[11] Skywork AI. *Matrix-Game: Interactive World Foundation Model.* arXiv:2506.18701, 2025. Weights & code MIT (`SkyworkAI/Matrix-Game`).

[12] J. Guo et al. *MineWorld: a Real-Time and Open-Source Interactive World Model on Minecraft.* arXiv:2504.08388, 2025. MIT (`microsoft/mineworld`).

[13] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, F. Fleuret. *Diffusion for World Modeling: Visual Details Matter in Atari (DIAMOND).* NeurIPS, 2024. MIT (`eloialonso/diamond`).

[14] J. Xiang et al. *Structured 3D Latents for Scalable and Versatile 3D Generation (TRELLIS).* CVPR, 2025. MIT (`microsoft/TRELLIS`).

[15] ACE Studio and StepFun. *ACE-Step: A Step Towards Music Generation Foundation Model.* 2025. Apache-2.0 (`ACE-Step/ACE-Step-v1-3.5B`).

[16] Wan Team, Alibaba Tongyi Lab. *Wan: Open and Advanced Large-Scale Video Generative Models (Wan 2.2).* 2025. Apache-2.0.

[17] O. Sitton (madebyollin). *TAESD: Tiny AutoEncoder for Stable Diffusion.* Open-source project, 2023.

[18] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models.* CVPR, 2022.

[19] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding.* ICML, 2023.

[20] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling.* arXiv:2302.01318, 2023.

[21] Y. W. Bernier. *Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization.* Game Developers Conference (Valve Software), 2001.

[22] Coalition for Content Provenance and Authenticity. *C2PA Technical Specification.* Content provenance standard.